

TypeScript サーバーサイドエンジニアが関数型から学 ぶべき 3 つのアイデア

@TSKaigi 2025 After Night

自己紹介

名前：majimaccho

お仕事: @caddi

職種: Web App Engineer

X: @majimaccho_



CADDi

TSKaigi お疲れ様でした！



TsKaigi 2025

レバレジーズトラック

LT

DAY2

14:00 ~ 14:30

Result型、自前で書くか、ライブラリ使うか



majimaccho

落選したプロポーズに書いた
もっと話したかったことを話します

TSKaigi 2024・2025 に参加して思いました

TSKaigi 2025

アセンドトラック

セッション

DAY1

16:30 ~ 17:00

fast-checkとneverthrowのPBT+Result型で堅
牢なビジネスロジックを実現する



上田慶祐

TSKaigi 2025

アセンドトラック

セッション

DAY2

11:30 ~ 12:00

Pragmatic Functional Programming in
TypeScript



yasaichi

複雑なビジネスルールに挑む：
正確性 と **効率性** を両立する

 **fp-ts**のチーム活用術

株式会社カケハシ

kosui (岩佐 幸翠)

TypeScript 関数型スタイルでバックエンド開発のリアル

株式会社 一休
伊藤 直也

2024/5/11 TSKaigi 2024

みんな関数型好き～～～（私も好き）

しかし

TypeScript は関数型言語ではありません

TypeScript は関数型言語ではない

- React をはじめフロントエンドでは関数型言語のプラクティスが行き入れられている
- 型の表現力の高さは TypeScript の美点で、現代の関数型言語のプラクティスと相性が良い
- ⚠ 一方で、現代の関数型言語に当たり前にある機能が TypeScript にはない。そのため、素の TypeScript に関数型言語のプラクティスを全て取り入れることはできない。

何を取り入れるかのベストプラはまだ定まっていない

そもそも関数型を取り入れること自体が
ベストプラクティスなのかも合意されていないと思われる

今回は TypeScript の言語仕様に相性が良いものに限って
取り入れる価値のあるアイデアを紹介します

TypeScript サーバーサイドエンジニアが関数型から学ぶべき 3 つのアイデア

- Make illegal states unrepresentable (不正な状態を定義できないようにする)
- Parse, don't validate (バリデーションするな、パースせよ)
- Result 型

Make illegal states unrepresentable

(不正な状態を定義できないようにする)

- 書籍、関数型ドメインモデリングで有名になった考え方
- 内部状態の組み合わせを隠蔽しない
- クラスではロジック+テストでこれを表現するが関数型のアプローチでは代数的データ型で表現する

Domain Modeling Made Functional

関数型 ドメインモデリング

ドメイン駆動設計とF#で
ソフトウェアの複雑さに立ち向かおう



ASCII
DWANGO

Scott Wlaschin 著
猪股 健太郎 訳

Make illegal states unrepresentable

連絡先（ContactInfo）はメールアドレスか郵送先の

どちらか、または両方を持つことができる

どちらも持たない状態は許容されないというルールを型で表現する

```
type EmailOnlyContactInfo = { type: "EmailOnly"; email: EmailContactInfo };
type PostalOnlyContactInfo = { type: "PostalOnly"; postal: PostalContactInfo };
type EmailAndPostalContactInfo = {
  type: "EmailAndPostal";
  email: EmailContactInfo;
  postal: PostalContactInfo;
};

type ContactInfo =
  | EmailOnlyContactInfo
  | PostalOnlyContactInfo
  | EmailAndPostalContactInfo;
```

Make illegal states unrepresentable は AI に優しい(mizchi さんの発表から)

効くプロンプト: TS + 関数型ドメインモデリング

TypeScript で関数型ドメインモデリングを行う。class を使わず関数による実装を優先する。
代数的データでドメインをモデリングする。

出力例

```
```ts
type FetchResult<T, E> = {
 ok: true;
 data: T
} | {
 ok: false;
 error: E
}
```
```

- (好みの問題はあるが) 状態の発散を抑えるのに不変データ構造が望ましい
- 関数型ドメインモデリング ドメイン駆動設計とF#でソフトウェアの複雑さに立ち向かおう | Scott Wlaschin, 猪股 健太郎

振る舞いについても型で表現する

メールを送信する関数は、メールアドレスが存在する場合にのみ呼び出せる
ContactInfo の変更時にも型で影響の有無が明示される

```
const sendEmail = async (  
  // PostalOnlyContactInfoは許容しない  
  contact: EmailOnlyContactInfo | EmailAndPostalContactInfo  
): Promise<EmailSendResult> => {  
  // ここでEmailが存在することを型で保証されているので  
  // 検証ロジックは不要  
};
```

Parse, don't validate (バリデーションするな、パースせよ)

- Always Valid Domain Model / セキュアバイデザインでも類似した考え方がある
- 外界からの信頼できない入力可能な限り外側の層で信頼可能な値に Parse する
- Unvalidated (未検証) と Validated (検証済) 明確に型レベルで区別する
- 不正な状態は存在してはいけないとすることで後続コードは全てドメインモデルを信じることができる

Parse, don't validate ではない例

```
const validateEmail = (emailStr: string): boolean => {  
    // 無効なメールアドレスであれば false を返す  
    // 有効なメールアドレスであれば true を返す  
};  
  
const sendEmail = async (emailStr: string) => {  
    if (!validateEmail(emailStr)) {  
        throw new InvalidEmailAddressError(emailStr);  
    }  
  
    // このままでは EmailService.sendはemailStr が不正な値である可能性がある  
    await EmailService.send(emailStr);  
};
```

Parse, don't validate の例

```
const parseEmail = (  
  unvalidatedEmail: string  
) :  
  | { isOk: true; value: ValidatedEmail }  
  | { isOk: false; error: InvalidEmailFormatError } => {  
    // 無効なメールアドレスであれば  
    // { isOk: false, error: InvalidEmailFormatError } を返す  
    // 有効なメールアドレスであれば  
    // { isOk: true, value: ValidatedEmail } を返す  
  };  
  
const sendEmail = async (unvalidatedEmail: string) => {  
  const parseResult = parseEmail(unvalidatedEmail);  
  
  if (!parseResult.isOk) {  
    return parseResult.error;  
  }  
  // EmailService.sendは引数の値を信用できるため再度検証する必要はない  
  await EmailService.send(parseResult.value);  
};
```

宣伝：翻訳したよ

翻訳: Parse, don't validate (バリデーションせずパースせよ)



まじまっちょ

Zenn

Result 型

- TS の try-catch はエラーを unknown にする
- どの関数が throw するのかインターフェースから理解不能

さっきの Parse, don't validate の例は Result 型を返している

```
const parseEmail = (  
  unvalidatedEmail: string  
) :  
| { isOk: true; value: ValidatedEmail }  
| { isOk: false; error: InvalidEmailFormatError } => {...};  
  
const sendEmail = async (unvalidatedEmail: string) => {  
  const parseResult = parseEmail(unvalidatedEmail);  
  
  if (!parseResult.isOk) {  
    return parseResult.error;  
  }  
  
  await EmailService.send(parseResult.value);  
};
```

型定義から起こりうるエラーが明示されている

```
const parseEmail = (  
  unvalidatedEmail: string  
) :  
  | { isOk: true; value: ValidatedEmail }  
  | { isOk: false; error: InvalidEmailFormatError } => {  
    // 無効なメールアドレスであれば  
    // { isOk: false, error: InvalidEmailFormatError } を返す  
    // 有効なメールアドレスであれば  
    // { isOk: true, value: ValidatedEmail } を返す  
  };
```

呼び出し時のエラーハンドリングも改善される

起こりうるエラーが明示的なのでエラー型ごとに個別に対処できる
Result 型

```
const sendEmail = async (unvalidatedEmail: string) => {  
  const parseResult = parseEmail(unvalidatedEmail);  
  
  if (!parseResult.isOk) {  
    // isOkがfalseの場合が起こりうるので処理が必要であることが明示的  
    return parseResult.error;  
  }  
  
  await EmailService.send(parseResult.value);  
};
```

汎用 Result 型を定義する

```
type Result<T, E> =  
  | { isOk: true; value: T }  
  | { isOk: false; error: E };  
  
// Result 型を使って parseEmail を定義する  
const parseEmail = (  
  unvalidatedEmail: string  
) : Result<ValidatedEmail, InvalidEmailFormatError> => {...}
```

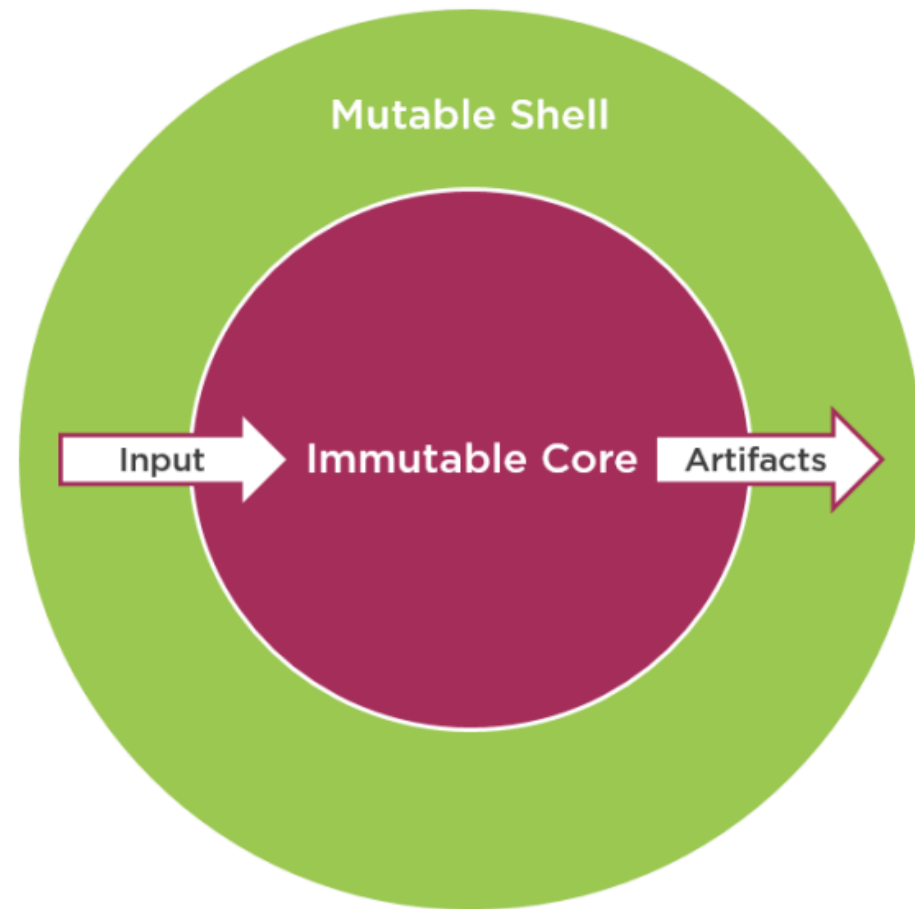
??? 「今までのプログラミングと違いすぎて採用できないよ」

じゃあ段階的に取り入れましょう 🐱

関数型のアイディアの段階的採用

ドメイン層から始める

- 関数型のアイディアは副作用のない世界ほど恩恵が大きい
- ドメイン層は副作用のあるコードがあるべきではないのでドメイン層から始める



関数型のアイデアの段階的採用

Parse, don't validate から始める

- Parse, don't validate 自体は関数型コミュニティでのスローガンだが、コアとなる考え方は Always Valid Domain Model / セキュアバイデザインに近い
- クラス指向的な設計であっても取り入れやすいし部分的にも適用可能
- クラスを使わない表現や Result 型は統一感がないと混乱を招きやすい

ご清聴ありがとうございました